



ROUTERS LINUX

PAR P604



11 DE ABRIL DE 2026

ANDRÉS VIZCAÍNO BAZAGA

DIRIGIDO AL PROFESOR DON ALFREDO ABAD DOMINGO

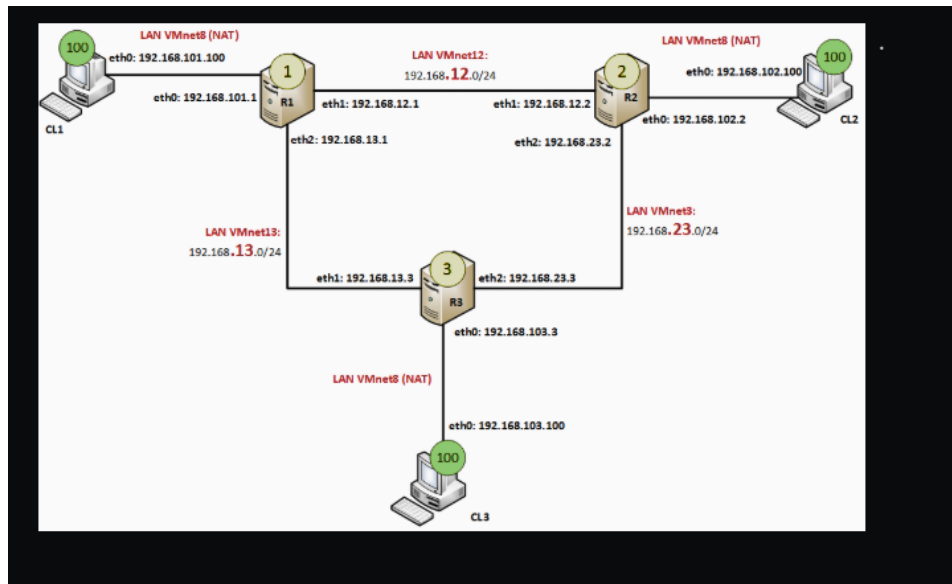
Índice

Introducción	2
Paso1: Configuración de las redes virtuales en VMware	4
Paso 2: Configuración de las Direcciones IP y Activación del Enrutamiento	6
Paso 3: Implementación del Enrutamiento Estático en Anillo	11
Paso 6: Enrutamiento Dinámico Link-State (FRRouting con OSPF)	15
Paso 7: Configuración de RIPv2 (Routing Information Protocol).....	18
Conclusión	21

Introducción

En esta práctica, vamos a desplegar y configurar una topología de red en anillo compuesta por tres enrutadores y tres equipos cliente, utilizando para ello sistemas operativos basados en GNU/Linux. Todo este entorno de laboratorio lo virtualizaremos utilizando la herramienta VMware Workstation Pro.

Nuestro objetivo principal es simular una infraestructura de red donde configuraremos el direccionamiento IP de todas las interfaces e implementaremos el enrutamiento necesario para garantizar la conectividad total entre los distintos extremos del anillo. Para llevar a cabo este proyecto, emplearemos máquinas virtuales con Ubuntu Server 24.04 para los routers y Ubuntu Desktop 24.04 para los equipos cliente.



```
CL1 192.168.101.100 R1 192.168.101.1
CL2 192.168.102.100 R2 192.168.102.2
CL3 192.168.103.100 R3 192.168.103.3

R1 ens33 192.168.101.1
   ens34 192.168.12.1
   ens35 192.168.13.1

R2 ens33 192.168.102.1
   ens34 192.168.12.2
   ens35 192.168.23.2

R3 ens33 192.168.103.3
   ens34 192.168.13.3
   ens35 192.168.23.3
```

Las **LAN** se llamarán de la siguiente manera:

- CL1-R1 192.168.101.0
- CL2-R2 192.168.102.0
- CL3-R3 192.168.103.0
- R1-R2 192.168.12.0
- R2-R3 192.168.23.0
- R1-R3 192.168.13.0

Paso1: Configuración de las redes virtuales en VMware

Para preparar nuestra topología en VMware Workstation Pro, lo primero que **hacemos** es configurar la infraestructura de red de nuestras máquinas virtuales. En lugar de utilizar las redes virtuales tradicionales (VMnets), **hemos optado** por emplear la característica **LAN Segments**. Esta opción nos permite crear dominios de difusión (switches virtuales) totalmente aislados de forma rápida y sin generar adaptadores virtuales innecesarios en nuestro equipo físico.

Dado que tenemos tres redes locales y tres enlaces punto a punto para formar el anillo, **creamos** un total de 6 LAN Segments. A continuación, **asignamos** las tarjetas de red de cada máquina virtual a su segmento correspondiente de la siguiente manera:

En los Enrutadores (3 tarjetas cada uno):

- **Router 1:** * Adaptador 1: CL1-R1
 - Adaptador 2: R1-R2
 - Adaptador 3: R1-R3
- **Router 2:**
 - Adaptador 1: CL2-R2
 - Adaptador 2: R1-R2
 - Adaptador 3: R2-R3
- **Router 3:**
 - Adaptador 1: CL3-R3
 - Adaptador 2: R1-R3
 - Adaptador 3: R2-R3

En los Clientes (1 tarjeta cada uno):

- **Cliente 1:** CL1-R1
- **Cliente 2:** CL2-R2
- **Cliente 3:** CL3-R3

Con esta configuración, **garantizamos** que el cableado virtual de nuestra topología en anillo esté perfectamente estructurado antes de encender los equipos.

Paso 2: Configuración de las Direcciones IP y Activación del Enrutamiento

Una vez configurados los segmentos de red virtual y encendidos los equipos, **procedemos** a configurar las interfaces de red y a habilitar las funciones de enrutamiento en el sistema operativo.

2.1. Activación del IP Forwarding (Solo en R1, R2 y R3)

Dado que el núcleo de GNU/Linux bloquea el tráfico entre distintas tarjetas de red por motivos de seguridad, **necesitamos** habilitar el reenvío de paquetes (IP Forwarding) en nuestros tres servidores. Para ello, **realizamos** los siguientes pasos en cada uno de los enrutadores:

1. **Abrimos** el archivo de variables del sistema en el editor de texto Nano ejecutando el comando: `sudo nano /etc/sysctl.conf`
2. **Buscamos** la línea `#net.ipv4.ip_forward=1` y le **borramos** el símbolo de almohadilla (#) del principio para descomentarla.
3. **Guardamos** los cambios (Ctrl+O, Enter) y **cerramos** el editor (Ctrl+X).
4. **Aplicamos** la configuración en caliente para que el sistema asuma su rol de enrutador sin necesidad de reiniciar, usando el comando: `sudo sysctl -p`

```
# /etc/sysctl.conf - Configuration file for setting system variables
# See /etc/sysctl.d/ for additional system variables.
# See sysctl.conf (5) for information.
#

#kernel.domainname = example.com

# Uncomment the following to stop low-level messages on console
#kernel.printk = 3 4 1 3

#####
# Functions previously found in netbase
#

# Uncomment the next two lines to enable Spoof protection (reverse-path filter)
# Turn on Source Address Verification in all interfaces to
# prevent some spoofing attacks
#net.ipv4.conf.default.rp_filter=1
#net.ipv4.conf.all.rp_filter=1

# Uncomment the next line to enable TCP/IP SYN cookies
# See https://lwn.net/Articles/277146/
# Note: This may impact IPv6 TCP sessions too
#net.ipv4.tcp_syncookies=1

# Uncomment the next line to enable packet forwarding for IPv4
net.ipv4.ip_forward=1

# Uncomment the next line to enable packet forwarding for IPv6
# Enabling this option disables Stateless Address Autoconfiguration
# based on Router Advertisements for this host
#net.ipv6.conf.all.forwarding=1

#####
# Additional settings - these settings can improve the network
# security of the host and prevent against some network attacks
# including spoofing attacks and man in the middle attacks through
# redirection. Some network environments, however, require that these
# settings are disabled so review and enable them as needed.
#

# Do not accept ICMP redirects (prevent MITM attacks)
#net.ipv4.conf.all.accept_redirects = 0
#net.ipv4.conf.default.accept_redirects = 0
#_or_

router1@router1:~$ sudo sysctl -p
net.ipv4.ip_forward = 1
router1@router1:~$
```

2.2. Configuración del direccionamiento con Netplan

A continuación, **procedemos** a configurar el direccionamiento IP estático. En Ubuntu 24.04, esta gestión se realiza mediante la utilidad **Netplan**, que emplea archivos con formato YAML. En cada una de las 6 máquinas, **llevamos a cabo** el siguiente procedimiento:

1. **Accedemos** a la configuración ejecutando `sudo nano /etc/netplan/` y utilizando la tecla Tabulador para autocompletar el nombre exacto del archivo .yaml que genera el sistema.
2. **Borramos** el contenido por defecto y **redactamos** la nueva configuración, asegurándonos de respetar estrictamente la indentación con espacios.

Para los **equipos cliente**, **establecemos** su dirección IP y la puerta de enlace predeterminada apuntando a su enrutador local:

Configuración que aplicamos al Cliente 1 (CL1)

network:

version: 2

renderer: NetworkManager

ethernets:

ens33:

dhcp4: false

addresses: [192.168.101.100/24]

routes:

- to: default

via: 192.168.101.1



```

avb1@avb1-VMware-Virtual-Platform: ~
GNU nano 7.2 /etc/netplan/01-network-manager-all.yaml
# Let NetworkManager manage all devices on this system
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    ens33:
      dhcp4: false
      addresses: [192.168.101.100/24]
      routes:
        - to: default
          via: 192.168.101.1
  
```

Configuración que aplicamos al Cliente 2 (CL2)

network:

version: 2

renderer: NetworkManager

ethernets:

ens33:

dhcp4: false

addresses: [192.168.102.100/24]

routes:

- to: default

via: 192.168.102.1

```
Firefox
[1/2] /etc/netplan/01-network-manager-all.yaml
# Let NetworkManager manage all devices on this system
network:
  version: 2
  renderer: NetworkManager
  ethernet:
    ens33:
      dhcp4: false
      addresses: [192.168.102.100/24]
      routes:
        - to: default
          via: 192.168.102.1
```

Configuración que aplicamos al Cliente 3 (CL3)

network:

version: 2

renderer: NetworkManager

ethernets:

ens33:

dhcp4: false

addresses: [192.168.103.100/24]

routes:

- to: default

via: 192.168.103.3

```
GNU nano 7.2 /etc/netplan/01-network-manager-all.yaml *
# Let NetworkManager manage all devices on this system
network:
  version: 2
  renderer: NetworkManager
  ethernet:
    ens33:
      dhcp4: false
      addresses: [192.168.103.100/24]
      routes:
        - to: default
          via: 192.168.103.3
```

Para los **enrutadores**, **configuramos** sus tres interfaces (ens33 para la red local y ens34/ens35 para los enlaces del anillo):

Configuración que aplicamos al Router 1 (R1)

network:

version: 2

renderer: networkd

ethernets:

ens33:

dhcp4: false

addresses: [192.168.101.1/24]

```
GNU nano 7.2 /etc/netplan/50-cloud-init.yaml *
network:
  version: 2
  renderer: networkd
  ethernet:
    ens33:
      dhcp4: false
      addresses: [192.168.101.1/24]
    ens34:
      dhcp4: false
      addresses: [192.168.12.1/24]
    ens35:
      dhcp4: false
      addresses: [192.168.13.1/24]
```

ens34:

dhcp4: false

addresses: [192.168.12.1/24]

ens35:

dhcp4: false

addresses: [192.168.13.1/24]

Configuración que aplicamos al Router 2 (R2)

network:

version: 2

renderer: networkd

ethernets:

ens33:

dhcp4: false

addresses: [192.168.102.1/24]

ens34:

dhcp4: false

addresses: [192.168.12.2/24]

ens35:

dhcp4: false

addresses: [192.168.23.2/24]



```

GNU nano 7.2 /etc/netplan/50-cloud-init.yaml *
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: false
      addresses: [192.168.102.1/24]
    ens34:
      dhcp4: false
      addresses: [192.168.12.2/24]
    ens35:
      dhcp4: false
      addresses: [192.168.23.2/24]
  
```

Configuración que aplicamos al Router 3 (R3)

network:

version: 2

renderer: networkd

ethernets:

ens33:

dhcp4: false

addresses: [192.168.103.3/24]

ens34:

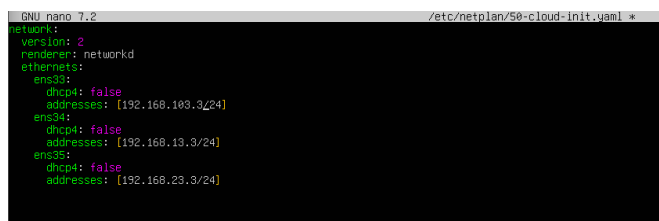
dhcp4: false

addresses: [192.168.13.3/24]

ens35:

dhcp4: false

addresses: [192.168.23.3/24]



```

GNU nano 7.2 /etc/netplan/50-cloud-init.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: false
      addresses: [192.168.103.3/24]
    ens34:
      dhcp4: false
      addresses: [192.168.13.3/24]
    ens35:
      dhcp4: false
      addresses: [192.168.23.3/24]
```

- Finalmente, tras guardar los cambios en todos los equipos, **aplicamos** la configuración de red ejecutando el comando:

- `sudo netplan apply`

Con este paso, **logramos** que la topología disponga de direccionamiento IP completo y conectividad básica entre dispositivos vecinos.

Paso 3: Implementación del Enrutamiento Estático en Anillo

Para lograr la conectividad total entre todos los extremos de nuestra topología, **necesitamos** configurar las tablas de enrutamiento en nuestros tres servidores.

Accedemos nuevamente al archivo de configuración de red de cada router. En cada interfaz que conecta con otro router, **añadimos** el bloque **routes**, indicando la red de destino (**to**) y la dirección IP del router vecino por el que se debe enviar el tráfico (**via**).

Configuramos el **Router 1 (R1)** para que alcance la LAN 2 a través de R2, y la LAN 3 a través de R3. Además, le enseñamos el camino hacia la red de enlace entre R2 y R3:

network:

version: 2

renderer: networkd

ethernets:

ens33:

dhcp4: false

addresses: [192.168.101.1/24]

ens34:

dhcp4: false

addresses: [192.168.12.1/24]

routes:

- to: 192.168.102.0/24

via: 192.168.12.2

- to: 192.168.23.0/24

via: 192.168.12.2

ens35:

```
GNU nano 7.2 /etc/netplan/50-cloud-init.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: false
      addresses: [192.168.101.1/24]
    ens34:
      dhcp4: false
      addresses: [192.168.12.1/24]
      routes:
        - to: 192.168.102.0/24
          via: 192.168.12.2
        - to: 192.168.23.0/24
          via: 192.168.12.2
    ens35:
      dhcp4: false
      addresses: [192.168.13.1/24]
      routes:
        - to: 192.168.103.0/24
          via: 192.168.13.3
```

dhcp4: false

addresses: [192.168.13.1/24]

routes:

- to: 192.168.103.0/24

via: 192.168.13.3

A continuación, **configuramos** el **Router 2 (R2)** para que alcance la LAN 1 a través de R1, y la LAN 3 a través de R3:

network:

version: 2

renderer: networkd

ethernets:

ens33:

dhcp4: false

addresses: [192.168.102.1/24]

ens34:

dhcp4: false

addresses: [192.168.12.2/24]

routes:

- to: 192.168.101.0/24

via: 192.168.12.1

- to: 192.168.13.0/24

via: 192.168.12.1

ens35:

dhcp4: false

addresses: [192.168.23.2/24]

routes:

```
GNU nano 7.2 /etc/netplan/50-cloud-init.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: false
      addresses: [192.168.102.1/24]
    ens34:
      dhcp4: false
      addresses: [192.168.12.2/24]
      routes:
        - to: 192.168.101.0/24
          via: 192.168.12.1
        - to: 192.168.13.0/24
          via: 192.168.12.1
    ens35:
      dhcp4: false
      addresses: [192.168.23.2/24]
      routes:
        - to: 192.168.103.0/24
          via: 192.168.23.3
```

- to: 192.168.103.0/24

via: 192.168.23.3

Finalmente, **configuramos** el **Router 3 (R3)** para que alcance la LAN 1 a través de R1, y la LAN 2 a través de R2:

network:

version: 2

renderer: networkd

ethernets:

ens33:

dhcp4: false

addresses: [192.168.103.3/24]

ens34:

dhcp4: false

addresses: [192.168.13.3/24]

routes:

- to: 192.168.101.0/24

via: 192.168.13.1

- to: 192.168.12.0/24

via: 192.168.13.1

ens35:

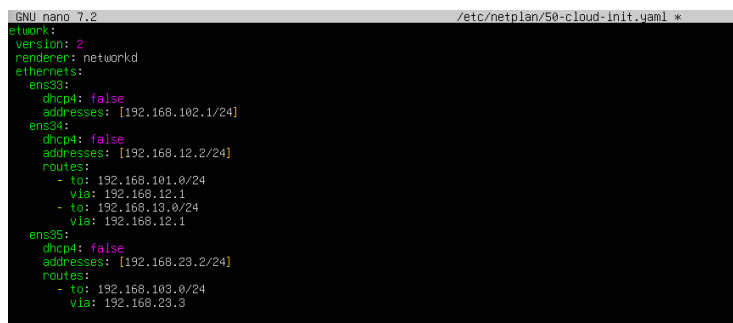
dhcp4: false

addresses: [192.168.23.3/24]

routes:

- to: 192.168.102.0/24

via: 192.168.23.2



```

GNU nano 7.2 /etc/netplan/50-cloud-init.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: false
      addresses: [192.168.103.3/24]
    ens34:
      dhcp4: false
      addresses: [192.168.13.3/24]
      routes:
        - to: 192.168.101.0/24
          via: 192.168.13.1
        - to: 192.168.12.0/24
          via: 192.168.13.1
    ens35:
      dhcp4: false
      addresses: [192.168.23.3/24]
      routes:
        - to: 192.168.102.0/24
          via: 192.168.23.2

```

Tras guardar las modificaciones en los tres servidores, **aplicamos** los cambios ejecutando el siguiente comando en cada uno de ellos:

- `sudo netplan apply`

Para culminar y validar la práctica, **realizamos** pruebas de conectividad (Ping) desde los equipos cliente para verificar que el enrutamiento funciona a través de todo el anillo.

Desde el terminal del **Cliente 1** (192.168.101.100), **lanzamos** pings hacia los clientes de las otras redes y comprobamos que obtenemos respuesta satisfactoria:

- `ping 192.168.102.100` (*Ping hacia CL2*)
- `ping 192.168.103.100` (*Ping hacia CL3*)

```
avb1@avb1-VMware-Virtual-Platform:~$ ping -c 4 192.168.103.100
PING 192.168.103.100 (192.168.103.100) 56(84) bytes of data.
64 bytes from 192.168.103.100: icmp_seq=1 ttl=62 time=1.24 ms
64 bytes from 192.168.103.100: icmp_seq=2 ttl=62 time=0.664 ms
64 bytes from 192.168.103.100: icmp_seq=3 ttl=62 time=0.981 ms
64 bytes from 192.168.103.100: icmp_seq=4 ttl=62 time=0.519 ms

--- 192.168.103.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3040ms
rtt min/avg/max/mdev = 0.519/0.850/1.239/0.279 ms
avb1@avb1-VMware-Virtual-Platform:~$ ping -c 4 192.168.102.100
PING 192.168.102.100 (192.168.102.100) 56(84) bytes of data.
64 bytes from 192.168.102.100: icmp_seq=1 ttl=62 time=1.12 ms
64 bytes from 192.168.102.100: icmp_seq=2 ttl=62 time=2.54 ms
64 bytes from 192.168.102.100: icmp_seq=3 ttl=62 time=2.23 ms
64 bytes from 192.168.102.100: icmp_seq=4 ttl=62 time=5.37 ms

--- 192.168.102.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 1.120/2.814/5.366/1.565 ms
```


Paso 3: Declaración de Redes en OSPF A continuación, entramos en la consola interactiva de FRR ejecutando `sudo vtysh`. En cada router, debemos declarar las redes que tiene conectadas directamente para que se las anuncie a sus vecinos dentro del Área 0 (el *backbone* de OSPF).

En el Router 1:

`configure terminal`

`router ospf`

`network 192.168.101.0/24 area 0`

`network 192.168.12.0/24 area 0`

`network 192.168.13.0/24 area 0`

`end`

`write memory`

```
router1@router1:~$ sudo systemctl restart frr
router1@router1:~$ sudo vtysh

Hello, this is FRRouting (version 8.4.4).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

router1# configure terminal
router1(config)# router ospf
router1(config-router)# network 192.168.101.0/24 area 0
router1(config-router)# network 192.168.12.0/24 area 0
router1(config-router)# network 192.168.13.0/24 area 0
router1(config-router)# end
router1# write memory
Note: this version of vtysh never writes vtysh.conf
Building Configuration...
Integrated configuration saved to /etc/frr/frr.conf
[OK]
router1#
```

En el Router 2:

`configure terminal`

`router ospf`

`network 192.168.102.0/24 area 0`

`network 192.168.12.0/24 area 0`

`network 192.168.23.0/24 area 0`

`end`

`write memory`

```
router2@router2:~$ sudo systemctl restart frr
router2@router2:~$ sudo vtysh

Hello, this is FRRouting (version 8.4.4).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

router2# configure terminal
router2(config)# router ospf
router2(config-router)# network 192.168.102.0/24 area 0
router2(config-router)# network 192.168.12.0/24 area 0
router2(config-router)# network 192.168.23.0/24 area 0
router2(config-router)# end
router2# write memory
Note: this version of vtysh never writes vtysh.conf
Building Configuration...
Integrated configuration saved to /etc/frr/frr.conf
[OK]
router2#
```

En el Router 3:

`configure terminal`

`router ospf`

`network 192.168.103.0/24 area 0`

`network 192.168.13.0/24 area 0`

`network 192.168.23.0/24 area 0`

end

write memory

```
router3@router:~$ sudo systemctl restart frr
router3@router:~$ sudo vtysh

Hello, this is FRRouting (version 8.4.4).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

router# configure terminal
router(config)# router ospf
router(config-router)# network 192.168.103.0/24 area 0
router(config-router)# network 192.168.13.0/24 area 0
router(config-router)# network 192.168.23.0/24 area 0
router(config-router)# end
router# write memory
Note: this version of vtysh never writes vtysh.conf
Building Configuration...
Integrated configuration saved to /etc/frr/frr.conf
[OK]
router#
```

Paso 4: Comprobación Una vez guardado, podemos salir de la consola con **exit**. Para verificar que OSPF está funcionando, ejecutamos **ip route** en Linux. Deberíamos ver las rutas aprendidas dinámicamente marcadas con el protocolo **ospf**.

```
router1@router1:~$ ip route
default via 192.168.192.2 dev ens39 proto dhcp src 192.168.192.150 metric 100
192.168.12.0/24 dev ens37 proto kernel scope link src 192.168.12.1
192.168.13.0/24 dev ens38 proto kernel scope link src 192.168.13.1
192.168.23.0/24 nhid 33 proto ospf metric 20
    nexthop via 192.168.12.2 dev ens37 weight 1
    nexthop via 192.168.13.3 dev ens38 weight 1
192.168.101.0/24 dev ens33 proto kernel scope link src 192.168.101.1
192.168.102.0/24 nhid 28 via 192.168.12.2 dev ens37 proto ospf metric 20
192.168.103.0/24 nhid 34 via 192.168.13.3 dev ens38 proto ospf metric 20
192.168.192.0/24 dev ens39 proto kernel scope link src 192.168.192.150 metric 100
192.168.192.2 dev ens39 proto dhcp scope link src 192.168.192.150 metric 100
router1@router1:~$
```

```
router2@router2:~$ ip route
default via 192.168.192.2 dev ens39 proto dhcp src 192.168.192.151 metric 100
192.168.12.0/24 dev ens37 proto kernel scope link src 192.168.12.2
192.168.13.0/24 nhid 35 proto ospf metric 20
    nexthop via 192.168.12.1 dev ens37 weight 1
    nexthop via 192.168.23.3 dev ens38 weight 1
192.168.23.0/24 dev ens38 proto kernel scope link src 192.168.23.2
192.168.101.0/24 nhid 28 via 192.168.12.1 dev ens37 proto ospf metric 20
192.168.102.0/24 dev ens33 proto kernel scope link src 192.168.102.1
192.168.103.0/24 nhid 36 via 192.168.23.3 dev ens38 proto ospf metric 20
192.168.192.0/24 dev ens39 proto kernel scope link src 192.168.192.151 metric 100
192.168.192.2 dev ens39 proto dhcp scope link src 192.168.192.151 metric 100
router2@router2:~$
```

```
router3@router:~$ ip route
default via 192.168.192.2 dev ens39 proto dhcp src 192.168.192.152 metric 100
192.168.12.0/24 nhid 37 proto ospf metric 20
    nexthop via 192.168.13.1 dev ens37 weight 1
    nexthop via 192.168.23.2 dev ens38 weight 1
192.168.13.0/24 dev ens37 proto kernel scope link src 192.168.13.3
192.168.23.0/24 dev ens38 proto kernel scope link src 192.168.23.3
192.168.101.0/24 nhid 28 via 192.168.13.1 dev ens37 proto ospf metric 20
192.168.102.0/24 nhid 38 via 192.168.23.2 dev ens38 proto ospf metric 20
192.168.103.0/24 dev ens33 proto kernel scope link src 192.168.103.3
192.168.192.0/24 dev ens39 proto kernel scope link src 192.168.192.152 metric 100
192.168.192.2 dev ens39 proto dhcp scope link src 192.168.192.152 metric 100
router3@router:~$
```

Además, el comando ping entre los clientes de los extremos debería ser exitoso tras unos segundos de convergencia.

```
avb1@avb1-VNware-Virtual-Platform: $ ping -c 4 192.168.102.100
PING 192.168.102.100 (192.168.102.100) 56(84) bytes of data.
64 bytes from 192.168.102.100: icmp_seq=1 ttl=62 time=1.91 ms
64 bytes from 192.168.102.100: icmp_seq=2 ttl=62 time=5.19 ms
64 bytes from 192.168.102.100: icmp_seq=3 ttl=62 time=3.16 ms
64 bytes from 192.168.102.100: icmp_seq=4 ttl=62 time=2.32 ms

--- 192.168.102.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 1.910/3.145/5.193/1.265 ms
avb1@avb1-VNware-Virtual-Platform: $ ping -c 4 192.168.103.100
PING 192.168.103.100 (192.168.103.100) 56(84) bytes of data.
64 bytes from 192.168.103.100: icmp_seq=1 ttl=62 time=1.84 ms
64 bytes from 192.168.103.100: icmp_seq=2 ttl=62 time=3.89 ms
64 bytes from 192.168.103.100: icmp_seq=3 ttl=62 time=5.05 ms
64 bytes from 192.168.103.100: icmp_seq=4 ttl=62 time=6.39 ms

--- 192.168.103.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 1.843/4.293/6.391/1.668 ms
```

Paso 7: Configuración de RIPv2 (Routing Information Protocol)

Dado que ya hemos instalado y comprobado la suite **FRRouting (FRR)** en la práctica anterior de OSPF, los paquetes necesarios ya están en el sistema. El procedimiento ahora consiste en "apagar" el protocolo OSPF, "encender" el protocolo RIP, y configurar las redes correspondientes.

Paso 1: Cambio de Protocolo en los Demonios de FRR En los tres routers, debemos editar el archivo donde FRR gestiona qué protocolos están activos para hacer el cambio.

- `sudo nano /etc/frr/daemons`

Dentro del archivo, realizaremos dos modificaciones clave:

1. Buscamos la línea ospfd=yes y la cambiamos a **ospfd=no** (para detener OSPF).
2. Buscamos la línea ripd=no y la cambiamos a **ripd=yes** (para habilitar RIP).

Guardamos los cambios y, para que el sistema lea esta nueva configuración, reiniciamos el servicio de FRRouting:

- `sudo systemctl restart fr`

```
GNU nano 7.2 /etc/frr
```

This file tells the frr package which daemons to start.

Sample configurations for these daemons can be found in
/usr/share/doc/frr/examples/.

ATTENTION:

When activating a daemon for the first time, a config file, even if it is empty, has to be present & must be owned by the user and group "frr". else the daemon will not be started by /etc/init.d/frr. The permissions should be u=rw,g=,o=.

When using "vtysh" such a config file is also needed. It should be owned by group "frrvty" and set to u=rw,o= though. Check /etc/pam.d/frr, too.

The watchfrr, zebra and static daemons are always started.

```
igmpd=no  
ospfd=no  
ospf6d=no  
ripd=yes  
ripngd=no  
isisd=no  
l2isnd=no  
l2l3d=no  
lldpd=no  
nhrpd=no  
eigrpd=no  
babeld=no  
sharpd=no  
bfd=no  
rfsd=no  
fabricd=no  
vrpd=no  
battd=no
```

If this option is set the /etc/init.d/frr script automatically loads the config via "vtysh -b" when the servers are started.
Check /etc/pam.d/frr if you intend to use "vtysh"!

```
vtysh_enable=yes  
zebra_options="-A 127.0.0.1 -s 90000000"  
igmpd_options="-A 127.0.0.1"  
ospfd_options="-A 127.0.0.1"  
ospf6d_options="-A ::1"  
l2l3d_options="-A 127.0.0.1"
```

Paso 2: Declaración de Redes en RIP Una vez activo el demonio de RIP, entramos en la consola interactiva de FRR ejecutando `sudo vtysh` en cada router. Le indicaremos que queremos usar la **versión 2** de RIP y declararemos las redes conectadas directamente.

En el Router 1:

```
configure terminal
router rip
version 2
network 192.168.101.0/24
network 192.168.12.0/24
network 192.168.13.0/24
end
write memory
```

```
router1@router1:~$ sudo vtysh
Hello, this is FRRouting (version 8.4.4).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

router1# configure terminal
router1(config)# router rip
router1(config-router)# version 2
router1(config-router)# 192.168.101.0/24
% Unknown command: 192.168.101.0/24
router1(config-router)# network 192.168.101.0/24
router1(config-router)# network 192.168.12.0/24
router1(config-router)# network 192.168.13.0/24
router1(config-router)# end
router1# write memory
Note: this version of vtysh never writes vtysh.conf
Building Configuration...
Integrated configuration saved to /etc/frr/frr.conf
[OK]
router1# exit
router1@router1:~$
```

En el Router 2:

```
configure terminal
router rip
version 2
network 192.168.102.0/24
network 192.168.12.0/24
network 192.168.23.0/24
end
write memory
```

```
router2@router2:~$ sudo systemctl restart frr
router2@router2:~$ sudo vtysh
Hello, this is FRRouting (version 8.4.4).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

router2# configure terminal
router2(config)# router rip
router2(config-router)# version 2
router2(config-router)# network 192.168.102.0/24
router2(config-router)# network 192.168.12.0/24
router2(config-router)# network 192.168.13.0/24
router2(config-router)# end
router2# write memory
Note: this version of vtysh never writes vtysh.conf
Building Configuration...
Integrated configuration saved to /etc/frr/frr.conf
[OK]
router2# exit
router2@router2:~$
```

En el Router 3:

```
configure terminal
router rip
version 2
network 192.168.103.0/24
network 192.168.13.0/24
network 192.168.23.0/24
end
```

```
router3@router:~$ sudo systemctl restart frr
router3@router:~$ sudo vtysh
Hello, this is FRRouting (version 8.4.4).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

router# configure terminal
router(config)# router rip
router(config-router)# version 2
router(config-router)# network 192.168.103.0/24
router(config-router)# network 192.168.13.0/24
router(config-router)# network 192.168.23.0/24
router(config-router)# end
router# write memory
Note: this version of vtysh never writes vtysh.conf
Building Configuration...
Integrated configuration saved to /etc/frr/frr.conf
[OK]
router# exit
router3@router:~$
```

write memory

Paso 3: Verificación y Pruebas de Conectividad Con la configuración guardada, salimos de la consola con exit. Para confirmar que RIP está enrutando el tráfico correctamente:

1. Ejecutamos el comando `ip route` en Linux. Ahora deberíamos ver que las redes de destino han sido aprendidas dinámicamente y el sistema las marca bajo el protocolo RIP.

```
router1@router1:~$ ip route
default via 192.168.192.2 dev ens39 proto dhcp src 192.168.192.150 metric 100
192.168.12.0/24 dev ens37 proto kernel scope link src 192.168.12.1
192.168.13.0/24 dev ens38 proto kernel scope link src 192.168.13.1
192.168.23.0/24 nhid 58 via 192.168.13.3 dev ens38 proto rip metric 20
192.168.101.0/24 dev ens33 proto kernel scope link src 192.168.101.1
192.168.102.0/24 nhid 56 via 192.168.12.2 dev ens37 proto rip metric 20
192.168.103.0/24 nhid 58 via 192.168.13.3 dev ens38 proto rip metric 20
192.168.192.0/24 dev ens39 proto kernel scope link src 192.168.192.150 metric 100
192.168.192.2 dev ens39 proto dhcp scope link src 192.168.192.150 metric 100
router1@router1:~$ _
```

```
router2@router2:~$ ip route
default via 192.168.192.2 dev ens39 proto dhcp src 192.168.192.151 metric 100
192.168.12.0/24 dev ens37 proto kernel scope link src 192.168.12.2
192.168.13.0/24 nhid 90 via 192.168.12.1 dev ens37 proto rip metric 20
192.168.23.0/24 dev ens38 proto kernel scope link src 192.168.23.2
192.168.101.0/24 nhid 90 via 192.168.12.1 dev ens37 proto rip metric 20
192.168.102.0/24 dev ens33 proto kernel scope link src 192.168.102.1
192.168.103.0/24 nhid 90 via 192.168.12.1 dev ens37 proto rip metric 20
192.168.192.0/24 dev ens39 proto kernel scope link src 192.168.192.151 metric 100
192.168.192.2 dev ens39 proto dhcp scope link src 192.168.192.151 metric 100
router2@router2:~$ _
```

```
router3@router:~$ ip route
default via 192.168.192.2 dev ens39 proto dhcp src 192.168.192.152 metric 100
192.168.12.0/24 nhid 60 via 192.168.13.1 dev ens37 proto rip metric 20
192.168.13.0/24 dev ens37 proto kernel scope link src 192.168.13.3
192.168.23.0/24 dev ens38 proto kernel scope link src 192.168.23.3
192.168.101.0/24 nhid 60 via 192.168.13.1 dev ens37 proto rip metric 20
192.168.102.0/24 nhid 60 via 192.168.13.1 dev ens37 proto rip metric 20
192.168.103.0/24 dev ens33 proto kernel scope link src 192.168.103.3
192.168.192.0/24 dev ens39 proto kernel scope link src 192.168.192.152 metric 100
192.168.192.2 dev ens39 proto dhcp scope link src 192.168.192.152 metric 100
router3@router:~$ _
```

2. Finalmente, acudimos a las máquinas cliente (por ejemplo, desde el Cliente 1) y lanzamos un comando `ping` hacia el Cliente 2 (192.168.102.100) y el Cliente 3 (192.168.103.100). Al igual que ocurre con OSPF, es posible que tarde unos breves segundos en converger antes de mostrar respuesta.

```
avb1@avb1-VMware-Virtual-Platform:~$ ping -c 4 192.168.103.100
PING 192.168.103.100 (192.168.103.100) 56(84) bytes of data.
64 bytes from 192.168.103.100: icmp_seq=1 ttl=62 time=1.28 ms
64 bytes from 192.168.103.100: icmp_seq=2 ttl=62 time=1.03 ms
64 bytes from 192.168.103.100: icmp_seq=3 ttl=62 time=2.18 ms
64 bytes from 192.168.103.100: icmp_seq=4 ttl=62 time=3.12 ms

--- 192.168.103.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 1.026/1.904/3.124/0.824 ms
avb1@avb1-VMware-Virtual-Platform:~$ ping -c 4 192.168.102.100
PING 192.168.102.100 (192.168.102.100) 56(84) bytes of data.
64 bytes from 192.168.102.100: icmp_seq=1 ttl=62 time=1.66 ms
64 bytes from 192.168.102.100: icmp_seq=2 ttl=62 time=4.16 ms
64 bytes from 192.168.102.100: icmp_seq=3 ttl=62 time=2.54 ms
64 bytes from 192.168.102.100: icmp_seq=4 ttl=62 time=4.07 ms

--- 192.168.102.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 1.664/3.108/4.159/1.054 ms
avb1@avb1-VMware-Virtual-Platform:~$
```

Conclusión

En esta práctica se ha desplegado y comprobado con éxito una topología de red en anillo compuesta por tres enrutadores basados en Linux y sus respectivos equipos cliente. A lo largo del proyecto, se ha evolucionado desde una configuración inicial de enrutamiento estático puro hasta la implementación de protocolos de enrutamiento dinámico de estándar industrial.

Durante la primera fase, se consolidaron los conocimientos sobre la configuración de interfaces de red y tablas de rutas manuales mediante **Netplan**.

Posteriormente, el laboratorio avanzó hacia el enrutamiento dinámico utilizando la suite **FRRouting (FRR)**. En primer lugar, se implementó **OSPF**, lo que permitió observar en tiempo real la convergencia autónoma de la red, la formación de adyacencias entre routers vecinos y, de manera destacada, la capacidad del protocolo para realizar un balance de carga automático al disponer de caminos redundantes en la topología de anillo.

Finalmente, se realizó una transición fluida hacia **RIPv2**, permitiendo contrastar la configuración de un protocolo avanzado de estado de enlace (OSPF) frente a un protocolo clásico de vector-distancia (RIP).

En definitiva, este proyecto no solo ha servido para garantizar la conectividad total de extremo a extremo mediante pruebas ICMP (Ping), sino que ha proporcionado una experiencia práctica inestimable en la administración, configuración y depuración de redes IP complejas.